

Databricks Connector Refactoring Documentation

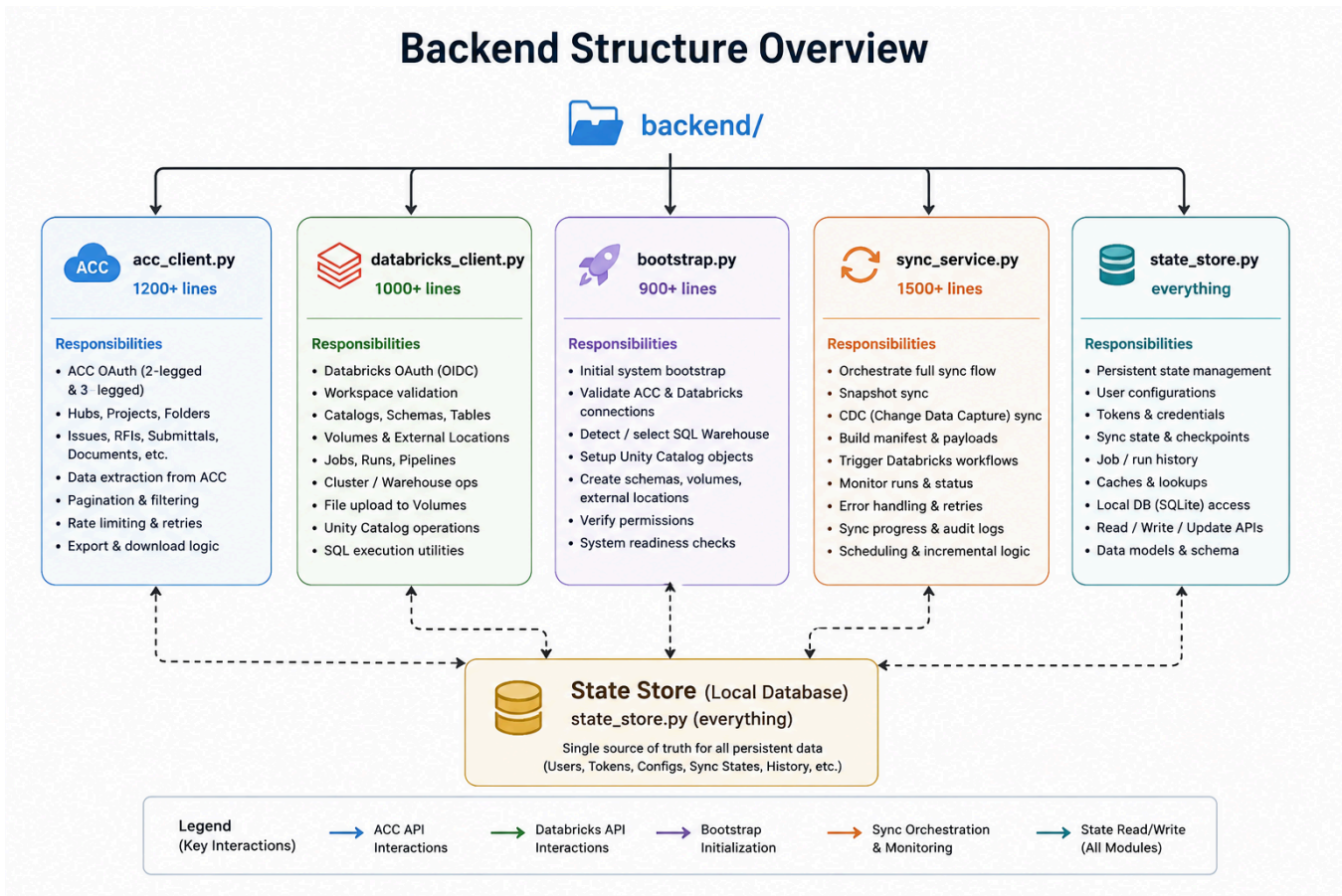
1. Objective

The objective of this refactoring was to improve the project structure by following SOLID principles and separating responsibilities into different modules. The functionality of the application remains unchanged while making the code easier to understand, maintain, and extend.

2. Old Project Structure

Before refactoring, the project contained several large files where multiple responsibilities were combined.

Example:



backend/

|

├─ acc_client.py (1200+ lines)

├─ databricks_client.py (1000+ lines)

├─ bootstrap.py (900+ lines)

├─ sync_service.py (1500+ lines)

└─ state_store.py (Everything)

acc_client.py – OAuth authentication, Access token management, Project APIs, Folder APIs, Data Connector APIs, Job monitoring, File downloads, HTTP request utilities, Error handling.

databricks_client.py – Databricks authentication, Workspace operations, Notebook uploads, Volume uploads, Pipeline management, Workflow creation, Job execution, Run monitoring, SQL Warehouse operations, File system utilities.

bootstrap.py – Initial setup, Unity Catalog creation, Schema and Volume creation, SQL Warehouse configuration, PK config upload, Notebook deployment, Pipeline creation, Workflow creation, Bootstrap state management.

sync_service.py – Complete sync orchestration, Snapshot sync, CDC sync, Data Connector request handling, Download management, Manifest generation, Schema processing, Registry management, Databricks uploads, Pipeline triggering, Workflow monitoring, Watermark updates, Sync status tracking, Error handling.

state_store.py – SQLite database management, Token storage, User configuration, Bootstrap state, Sync state, Watermarks, Pipeline IDs, Workflow IDs, Sync history, Application settings.

Problems

Problems

- Large monolithic files
 - Mixed responsibilities (SRP violation)
 - Difficult to maintain
 - Difficult to debug
 - Difficult to test
 - Duplicate code
 - Low modularity and code reusability
 - Hard to extend with new features
-

3. Refactoring Changes

Clients

Split `acc_client.py` into smaller modules:

- `auth_client.py`
- `project_client.py`
- `data_connector_client.py`
- `http_client.py`
- `constants.py`

`databricks_client.py` was kept as a single class because it maintains workspace state and handles all Databricks REST APIs.

Repository Layer

Split `state_store.py` into:

- `database.py`
- `token_repository.py`
- `config_repository.py`
- `bootstrap_repository.py`
- `sync_repository.py`
- `watermark_repository.py`
- `encryption.py`

`state_store.py` now acts as a facade that imports all repository functions.

Routes

Separated API endpoints into multiple route files:

- `acc_routes.py`
- `bootstrap_routes.py`
- `dashboard_routes.py`
- `databricks_routes.py`
- `sync_routes.py`

Each route handles only its related API endpoints.

Services

Split business logic into dedicated services.

Created:

- `bootstrap_service.py`
- `sync_service.py`
- `zerobus_service.py`

Sync logic was further divided into:

- `download_service.py`
 - `schema_service.py`
 - `registry_service.py`
 - `sync_orchestrator.py`
 - `sync_config.py`
-

Utilities

Created utility modules for reusable functionality:

- `auth.py`
 - `databricks_auth.py`
-

4. New Project Structure

```
backend/  
├── clients/  
├── repositories/  
├── routes/  
├── services/  
├── utils/  
├── config/  
└── models/
```

The project now follows a layered architecture where each folder has a clear responsibility.

5. SOLID Principles Applied

- **Single Responsibility Principle (SRP):** Large files were divided into focused modules.
 - **Open/Closed Principle (OCP):** New functionality can be added without modifying existing modules.
 - **Dependency Separation:** Business logic, database operations, API clients, and routes are separated.
-

6. Benefits

- Cleaner project structure
 - Easier debugging
 - Better readability
 - Easier maintenance
 - Better code reusability
 - Improved scalability
 - Better alignment with SOLID principles
-

7. Summary

The project was successfully refactored from a monolithic structure into a modular architecture. Responsibilities are clearly separated across Clients, Repositories, Routes, Services, and Utilities, making the application easier to develop, test, and maintain while preserving existing functionality.

Refactoring Documentation

Step 1: Project Structure Initialization

Objective

Reorganize the project into a modular folder structure by separating the UI and backend components.

Changes Made

UI Separation

Created dedicated folders for frontend resources:

```
static/  
├── app.css  
└── app.js
```

```
templates/  
└── index.html
```

- Moved CSS files to the **static** folder.
- Moved JavaScript files to the **static** folder.
- Placed the HTML page inside the **templates** folder following Flask conventions.

Backend Structure

Created a dedicated **backend** directory with separate folders for each application layer:

```
backend/  
├── clients/  
├── config/  
├── models/  
├── repositories/  
├── routes/  
├── services/  
└── utils/
```

Initially, these folders were created as the project foundation. Their implementations were added in later refactoring steps.

Benefits

- Separated frontend and backend code.
- Established a clean project structure.
- Prepared the application for modular development.
- Improved project organization and maintainability.

Refactoring Phase 2: Client Layer Modularization

Objective

Refactor the client layer by organizing all external API communication into a dedicated **clients** package and splitting the large `acc_client.py` into smaller, responsibility-based modules following the **Single Responsibility Principle (SRP)**.

Before Refactoring

```
backend/  
├── acc_client.py  
└── databricks_client.py
```

Problems

- Large client files with multiple responsibilities.
 - Difficult to maintain and debug.
 - High coupling between authentication, API calls, and business logic.
 - Hard to extend without modifying existing code.
-

Step 1 – Created the Clients Layer

Created a dedicated folder for all external API communication.

```
backend/  
└── clients/
```

Purpose

- Centralize all external API clients.
 - Separate API communication from business logic.
 - Improve project organization.
-

Step 2 – Moved Existing Client Files

Moved the existing client files into the new `clients` directory.

Before

```
backend/  
├── acc_client.py  
└── databricks_client.py
```

After

```
backend/  
├── clients/  
│   ├── acc_client.py  
│   └── databricks_client.py
```

Changes

- File locations updated.
- Project imports updated.

Example:

```
from backend import acc_client
```

became

```
from backend.clients import acc_client
```

No business logic was changed during this step.

Step 3 – Modularized **acc_client.py**

The large **acc_client.py** was split into multiple focused modules.

```
backend/  
├── clients/  
│   ├── acc_client.py  
│   └── acc/  
│       ├── auth_client.py  
│       ├── project_client.py  
│       ├── data_connector_client.py  
│       ├── http_client.py  
│       └── constants.py
```

Module Responsibilities

Module	Responsibility
<code>auth_client.py</code>	APS OAuth authentication and token management
<code>project_client.py</code>	Hub, Project and Folder APIs
<code>data_connector_client.py</code>	Autodesk Data Connector operations
<code>http_client.py</code>	Shared HTTP request helpers
<code>constants.py</code>	Shared API URLs and configuration constants

Step 4 – `acc_client.py` Became a Facade

Instead of changing every import across the project, `acc_client.py` was retained as the public entry point.

It now imports functionality from the smaller modules and exposes a single interface to the rest of the application.

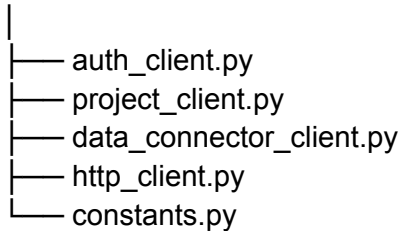
Routes



Services



`acc_client.py`



This allowed the rest of the application to continue using:

```
from backend.clients import acc_client
```

without requiring further code changes.

Step 5 – Databricks Client

The `databricks_client.py` file was moved into the `clients` layer.

```
backend/  
└── clients/  
    ├── acc_client.py  
    └── databricks_client.py
```

At this stage:

- File location was updated.
- Project imports were updated.
- Functionality remained unchanged.

This establishes the foundation for future modularization of the Databricks client.

Benefits

- Centralized external API clients.
- Split a large client into smaller, focused modules.
- Improved code organization and readability.
- Reduced coupling between different responsibilities.
- Easier debugging and maintenance.
- Better alignment with the **Single Responsibility Principle (SRP)**.
- Prepared the Databricks client for future modularization.

This is a polished, concise version of your documentation. It explains **Phase 3 to Phase 6** clearly without being overly long, making it suitable for a project report or GitHub documentation.

Phase 3 – Service Layer & Route Layer Refactoring

Objective

The original project contained business logic, Flask routes, API calls, and state management inside a few large files. This made the code difficult to understand, maintain, and extend.

The objective of this phase was to introduce a proper **Service Layer** and **Route Layer** by separating HTTP handling from business logic, following SOLID principles and a layered architecture.

Before Refactoring

backend/

bootstrap.py

- Flask endpoints
- Bootstrap logic
- Databricks operations
- State updates
- Response creation

sync_service.py

- Flask endpoints
- Sync orchestration
- Download logic
- Schema generation
- Registry updates
- Databricks workflow
- State management

Problems

- Business logic mixed with HTTP requests
- Large files with multiple responsibilities
- Difficult to debug and test
- Poor code organization
- Limited code reuse
- Violated Single Responsibility Principle (SRP)

What We Changed

Step 1 – Created Service Layer

Created a dedicated **services/** folder.

backend/

- services/
 - bootstrap_service.py
 - sync_service.py
 - zerobus_service.py

bootstrap_service.py

Moved all bootstrap-related business logic into a separate service.

Responsibilities:

- Workspace creation
- Catalog creation
- Schema creation
- Volume creation
- Notebook upload
- Pipeline creation
- Workflow creation
- Bootstrap configuration
- Bootstrap progress tracking

Flow:

Route



bootstrap_service



Databricks Client

sync_service.py

Created a dedicated synchronization service.

Responsibilities:

- Start sync
- Validate sync
- Execute sync
- Return sync status
- Coordinate synchronization

Later, this service was further modularized into the **services/sync/** package.

zerobus_service.py

Handles Zerobus-specific operations.

Responsibilities:

- Zerobus communication
 - Zerobus status
 - Helper methods
-

Step 2 – Created Route Layer

Introduced a dedicated **routes/** folder.

```
backend/  
└── routes/  
    ├── acc_routes.py  
    ├── bootstrap_routes.py  
    ├── dashboard_routes.py  
    ├── databricks_routes.py  
    ├── sync_routes.py  
    └── health_routes.py
```

Each route file now handles only HTTP requests and delegates business logic to services.

bootstrap_routes.py

- POST /bootstrap
- GET /bootstrap/status

Responsibilities:

- Receive request
 - Validate request
 - Call bootstrap_service
 - Return JSON response
-

sync_routes.py

- POST /sync
- GET /sync/status

Responsibilities:

- Receive request
 - Validate request
 - Call sync_service
 - Return response
-

acc_routes.py

Handles Autodesk ACC APIs.

Examples:

- GET /projects
- GET /folders
- GET /accounts

databricks_routes.py

Handles Databricks APIs.

Examples:

- GET /catalogs
- GET /schemas
- POST /test-connection

dashboard_routes.py

Dashboard-related endpoints.

Examples:

- GET /dashboard
- GET /state

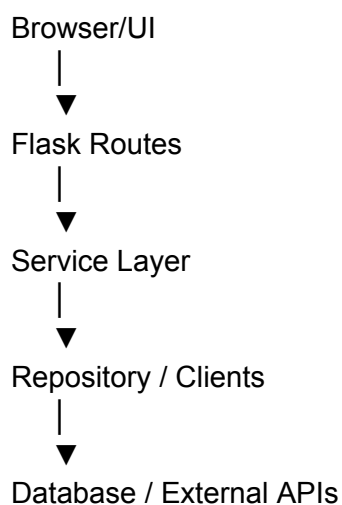
health_routes.py

Application health endpoint.

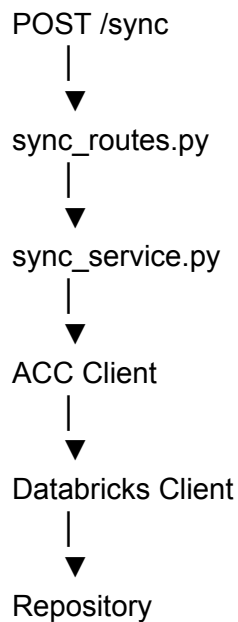
Example:

- GET /health

Request Flow



Example:



Benefits

- Clear separation of concerns
 - Cleaner route handlers
 - Easier maintenance
 - Better readability
 - Easier testing
 - Improved scalability
-

SOLID Principles Applied

- **SRP** – Routes handle HTTP, Services handle business logic, Clients handle external APIs, Repositories handle data access.
 - **OCP** – New features can be added by creating new services and routes.
 - **DIP** – Routes depend on services instead of implementing business logic directly.
-

Phase 4 – Repository Layer Refactoring (State Store Modularization)

Objective

The original `state_store.py` contained all database-related operations in one file, including database initialization, configuration, tokens, bootstrap information, watermarks, sync history, and encryption.

The goal was to modularize the repository layer while preserving the existing public API.

Before Refactoring

backend/

state_store.py

- |— Database initialization
- |— SQLite connection
- |— Encryption
- |— OAuth tokens
- |— Configuration
- |— Bootstrap data
- |— Watermarks
- |— Sync history
- |— Utilities

Problems

- Large monolithic file
 - Multiple responsibilities
 - Difficult to debug
 - Difficult to test
 - Poor maintainability
 - Violated SRP
-

What We Changed

Created a dedicated repository structure.

```
backend/
├── repositories/
│   ├── state_store.py
│   └── state/
│       ├── database.py
│       ├── encryption.py
│       ├── token_repository.py
│       ├── config_repository.py
│       ├── bootstrap_repository.py
│       ├── watermark_repository.py
│       └── sync_repository.py
```

Responsibilities

File	Responsibility
database.py	Database initialization and connection
encryption.py	Encryption and decryption
token_repository.py	OAuth token management
config_repository.py	Connector configuration
bootstrap_repository.py	Bootstrap information
watermark_repository.py	Watermark management
sync_repository.py	Sync history and status

state_store.py as Facade

Instead of removing `state_store.py`, it became a facade that re-exports functions from specialized repository modules.

Existing imports continue to work.

```
from backend.repositories import state_store
```

Benefits

- Smaller repository modules
- Easier debugging
- Better maintainability
- Easier testing
- Improved scalability
- Backward compatibility

SOLID Principles Applied

- **SRP** – Each repository manages one type of data.
 - **OCP** – New repositories can be added without modifying existing ones.
 - **DIP** – Services interact with the repository facade rather than low-level database implementation.
-

Phase 5 – Utility Layer Refactoring

Objective

Authentication helper functions were duplicated across routes, services, and clients. The goal was to centralize reusable authentication logic into a shared utility layer.

Before Refactoring

Authentication Logic

```
|— Routes
|— Services
|— Clients
|— Repeated code
```

Problems

- Duplicate code
 - Difficult maintenance
 - Authentication mixed with business logic
 - Poor reusability
-

What We Changed

Created a dedicated utility package.

```
backend/
├── utils/
│   ├── auth.py
│   └── databricks_auth.py
```

auth.py

Responsibilities:

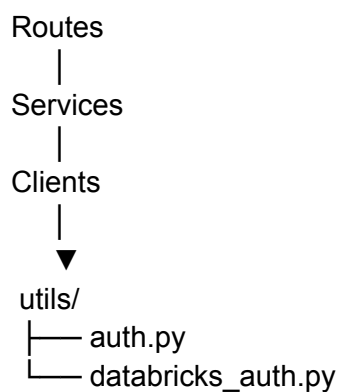
- User authentication
 - Session helpers
 - Auth validation
 - Shared authentication utilities
-

databricks_auth.py

Responsibilities:

- Databricks authentication
 - Token handling
 - Authorization headers
 - Shared Databricks auth helpers
-

Architecture



Benefits

- Centralized authentication
 - Better code reuse
 - Reduced duplication
 - Easier maintenance
 - Cleaner business logic
 - Easier testing
-

SOLID Principles Applied

- **SRP** – Each utility has one responsibility.
- **DRY** – Authentication logic is written once and reused everywhere.

Phase 6 – Sync Service Modularization

Objective

The original `sync_service.py` was approximately **1500+ lines** and contained the complete synchronization workflow.

The goal was to break this large service into smaller, focused modules while keeping the existing API unchanged.

Before Refactoring

```
backend/  
└── services/  
    └── sync_service.py  
        (~1500+ lines)
```

Problems

- Multiple responsibilities
 - Difficult to understand
 - Difficult to debug
 - Difficult to test
 - Poor maintainability
 - Violated SRP
-

What We Changed

Created a dedicated synchronization package.

```
backend/  
└── services/  
    ├── sync_service.py  
    └── sync/  
        ├── sync_orchestrator.py  
        ├── download_service.py  
        ├── schema_service.py  
        ├── registry_service.py  
        └── sync_config.py
```

sync_orchestrator.py

Responsibilities:

- Coordinate Snapshot Sync
 - Coordinate CDC Sync
 - Call download service
 - Call schema service
 - Call registry service
 - Trigger Databricks workflows
-

download_service.py

Responsibilities:

- Download Data Connector exports
 - Download ZIP files
 - Generate manifests
 - Upload files to Databricks Volume
-

schema_service.py

Responsibilities:

- Process schema metadata
 - Prepare schema artifacts
 - Support manifest generation
-

registry_service.py

Responsibilities:

- Primary key registry
 - Registry updates
 - Primary key mapping
-

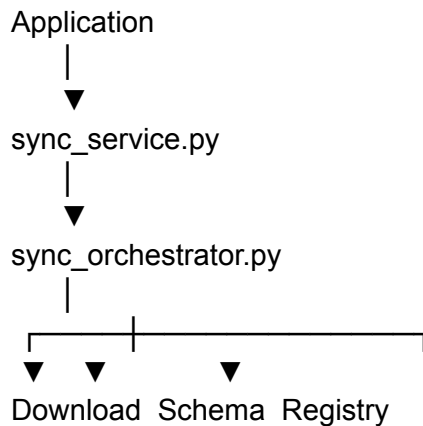
sync_config.py

Responsibilities:

- Sync configuration
- Constants
- Timeout settings
- Polling intervals

sync_service.py

The original file now acts as a **Facade**.



Existing imports remain unchanged.

```
from backend.services.sync_service import run_sync
```

Benefits

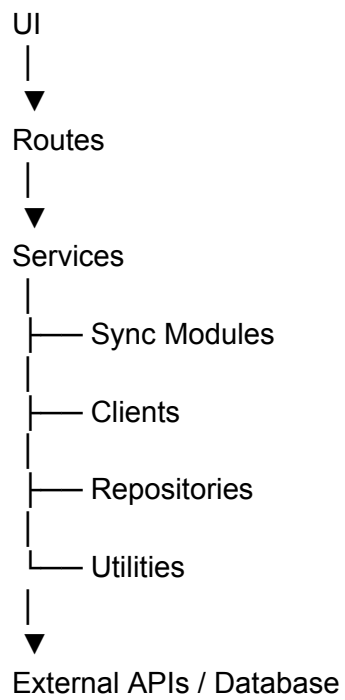
- Smaller modules
 - Easier debugging
 - Better readability
 - Easier testing
 - Reduced coupling
 - Better scalability
 - Improved maintainability
-

SOLID Principles Applied

- **SRP** – Each module has a single responsibility.
 - **OCP** – New synchronization features can be added without modifying existing modules.
 - **DIP** – The orchestrator coordinates specialized services instead of implementing everything itself.
-

Overall Result

By completing **Phases 3–6**, the project evolved from a monolithic architecture into a layered, modular design:



This refactoring significantly improved **maintainability, readability, scalability, testability, and adherence to SOLID principles**, while preserving backward compatibility through the use of **facade modules** such as `acc_client.py`, `state_store.py`, and `sync_service.py`.

Architecture Comparison

Before Refactoring (Monolithic Architecture)

The project consisted of a few large files where multiple responsibilities were combined into a single module.

```
acc-connector/
|
├── app.py
|
└── backend/
    |
    ├── acc_client.py      (1200+ lines)
    ├── databricks_client.py (1000+ lines)
    ├── bootstrap.py      (900+ lines)
    ├── sync_service.py    (1500+ lines)
    └── state_store.py     (Everything)
```

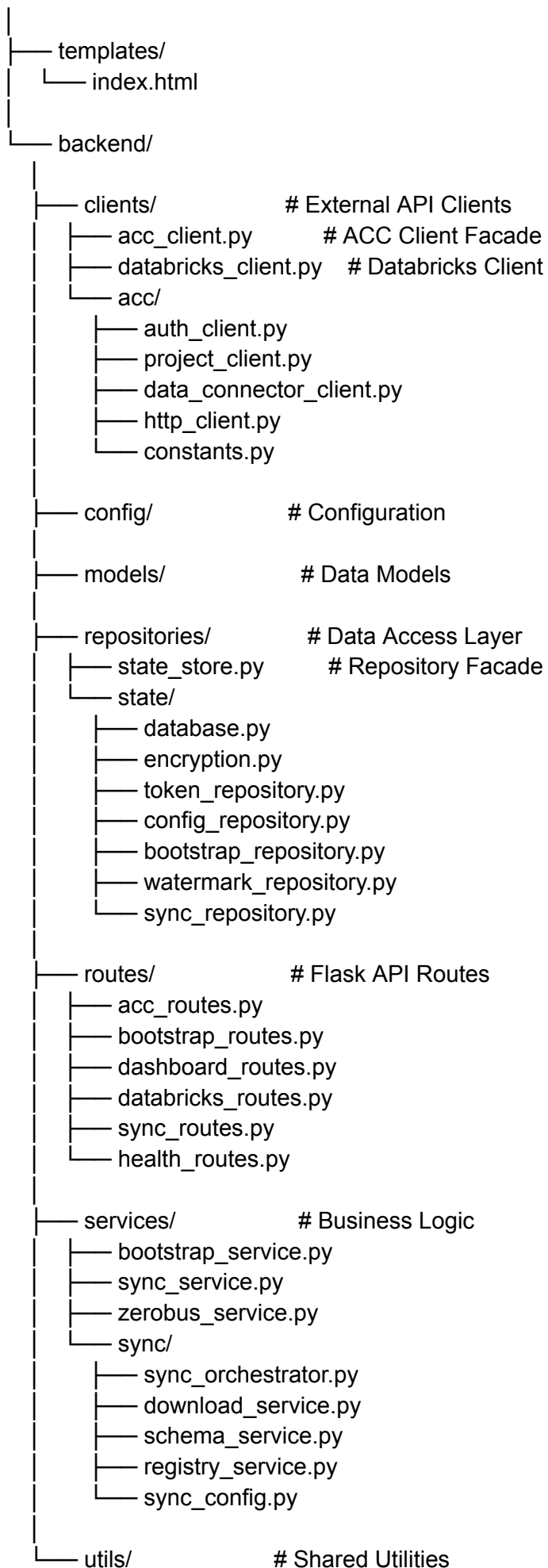
Characteristics

- Large monolithic files
 - Business logic mixed with API endpoints
 - Database logic in a single file
 - API client logic tightly coupled
 - Authentication code duplicated
 - Difficult to debug and test
 - Limited scalability
 - Poor separation of concerns
 - Multiple SOLID principle violations
-

After Refactoring (SOLID-Based Layered Architecture)

The project is now organized into clear layers, where each module has a single responsibility and follows a modular architecture.

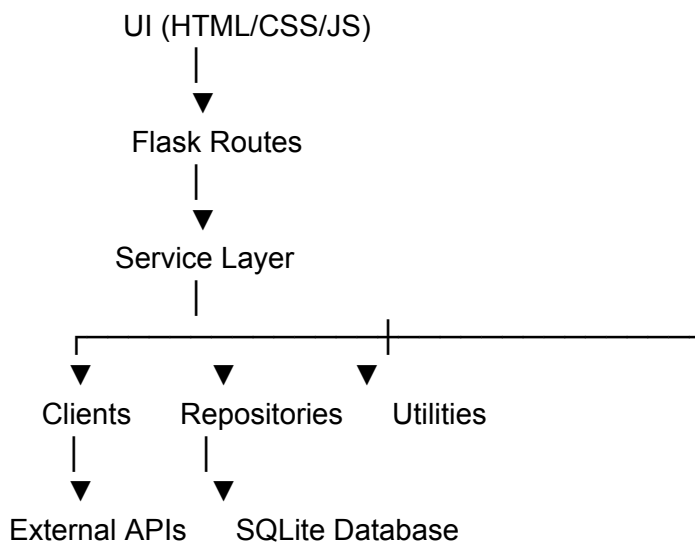
```
acc-connector/
|
├── app.py                # Flask application entry point
|
└── static/
    ├── app.css
    └── app.js
```



Old vs New Architecture

Area	Before Refactoring	After Refactoring
Architecture	Monolithic	Layered (SOLID-Based)
Project Structure	Few large files	Organized into folders and layers
API Clients	Single large client files	Modular client package with facades
Business Logic	Mixed with Flask routes	Dedicated Service Layer
HTTP Endpoints	Mixed with logic	Dedicated Route Layer
Database Access	Single <code>state_store.py</code>	Modular Repository Layer
Authentication	Duplicated across modules	Shared Utility Layer
Sync Processing	One 1500+ line file	Modular Sync Services
Code Reusability	Low	High
Maintainability	Difficult	Easy
Readability	Low	High
Testability	Difficult	Easier
Scalability	Limited	High
SOLID Compliance	Partial / Poor	Strong

Overall Architecture

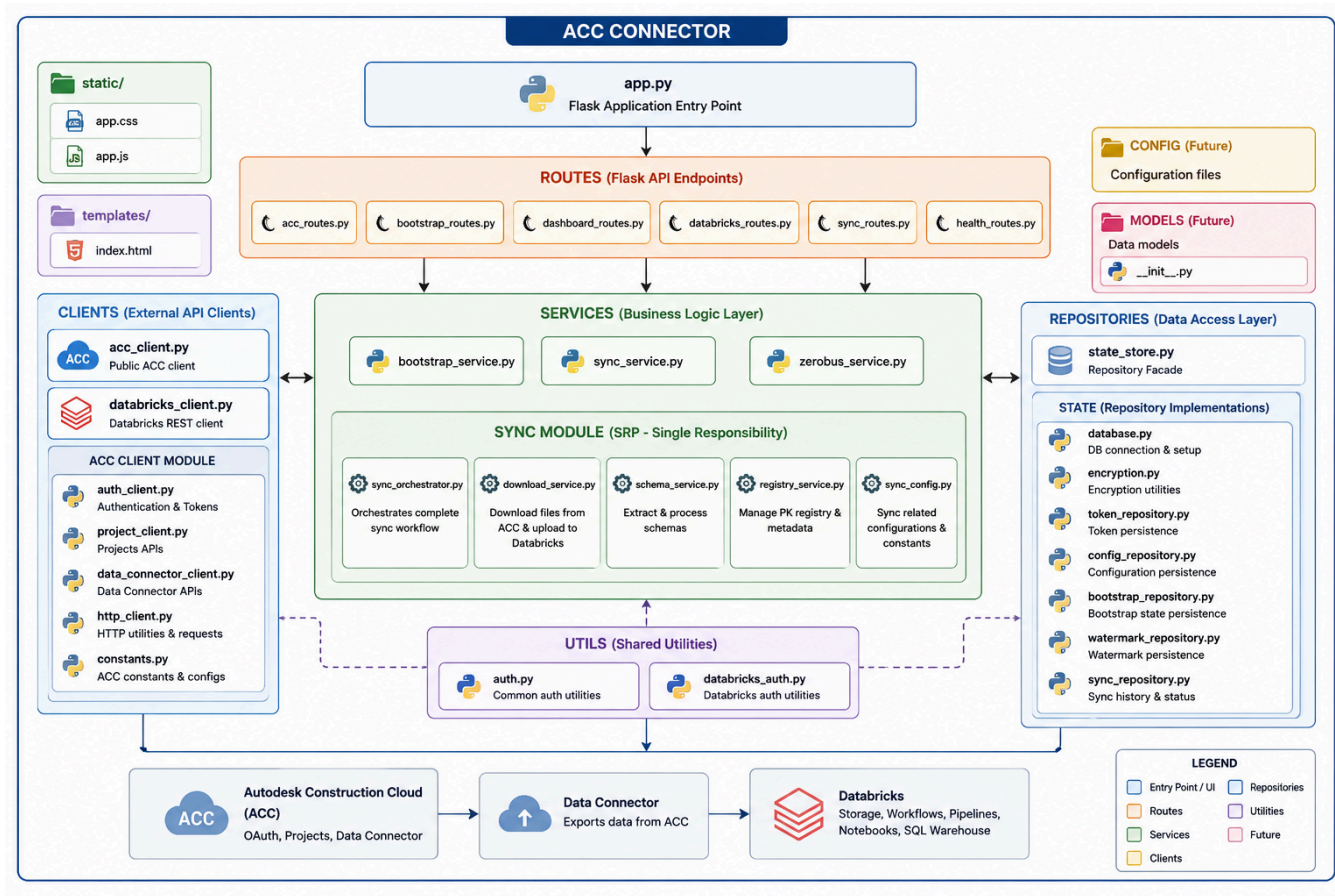


Refactoring Summary

Phase	Description
Phase 1	UI Separation (<code>static/</code> and <code>templates/</code>)
Phase 2	Client Layer Modularization (<code>clients/</code> and <code>clients/acc/</code>)
Phase 3	Service Layer & Route Layer Refactoring
Phase 4	Repository Layer Modularization (<code>state_store.py</code> → <code>repositories/state/</code>)
Phase 5	Utility Layer Refactoring (<code>utils/</code>)
Phase 6	Sync Service Modularization (<code>services/sync/</code>)

Final Result

The ACC Connector evolved from a **monolithic codebase** with a few large files into a **SOLID-based layered architecture**. Responsibilities are now clearly separated across **Routes, Services, Clients, Repositories, and Utilities**, making the project easier to understand, maintain, test, and extend while preserving backward compatibility through facade modules such as `acc_client.py`, `state_store.py`, and `sync_service.py`.



Work Items - BoardsData Connector - InsightACC Connector v2Jobs & Pipelines - Databricks

localhost:5000

Summarize

Chat

ACC ConnectorAutodesk Construction Cloud → Databricks

Start over

SETUP STEPS

Connect ACCAutodesk GAAuth

Connect DatabricksWorkshop workspace

Sync DataAll modules via Data Connector

DashboardSync history & analytics

Dashboard

DX_CCTech - Construction : Sample Project - Seaport Civic Center

This view summarizes **connector runs**. Data Connector bulk CSV exports landed in Unity Catalog Bronze SCD2 tables. Use [Databricks SQL](#) or [Lakehouse dashboards](#) for analytics on issues, RPA, Cost and other modules after sync. If Step 4 stayed locked after your first sync, refresh the page once so the app picks up completed runs.

263

CSV files (latest run)

Updated 23/6/2026, 3:58:57 pm

COMPLETE

Latest status

Pipeline finished

41m 47s

Duration

Last completed or failed run

100% (1 finished runs)

Pipeline reliability

Successful vs failed (recent window)

DATABRICKS LANDING ZONE

Workspace (aws): [Open workspace](#)

Unity Catalog: acc_connector_2_pk_test

Volume path: /volumes/acc_connector_2_pk_test/bronze/acc_bronze_volume

[Bronze pipeline](#)

EXPORT VOLUME TREND

CSV files landed per sync (on-demand and scheduled)

CSV files

RECENT OUTCOMES

Completed sync runs in the recent window

SucceededFailed

23°C

Light rain

Search

18:52

23-06-2026

Work Items - BoardsData Connector - InsightACC Connector v2Jobs & Pipelines - Databricks

localhost:5000

Summarize

Chat

DATABRICKS LANDING ZONE

Workspace (aws): [Open workspace](#)

Unity Catalog: acc_connector_2_pk_test

Volume path: /volumes/acc_connector_2_pk_test/bronze/acc_bronze_volume

[Bronze pipeline](#)

EXPORT VOLUME TREND

CSV files landed per sync (on-demand and scheduled)

CSV files

RECENT OUTCOMES

Completed sync runs in the recent window

SucceededFailed

Daily CDC Sync

No active periodic syncs.

[Enable Daily CDC Sync](#)

[Sync Now](#)

[Refresh](#)

Recent sync runs

RUN	TYPE	STARTED	STATUS	CSV FILES	DURATION
#1	Manual full	23/6/2026, 3:18:10 pm	Success	263	2507s

23°C

Light rain

Search

18:52

23-06-2026

Search data, notebooks, recent, and more...

CTRE V V

Jobs & Pipelines

Jobs & pipelinesRuns

Send feedback

Create new

Ingestion pipeline

Ingest data from apps, databases and files

ETL pipeline

Build ETL pipelines using SQL and Python

Job

Orchestrate notebooks, pipelines, queries and more

Filter by name or ID substring

AllJobsPipelinesOwned by meAccessible by meFavoritesTagsRun asCreate

Name	Type	Tags	Run as	Trigger	Recent runs	
CCTech_ACCConnector_Bronze	Pipeline - ETL		Gajendra tandekar			
CCTech_ACCConnector_Bronze_CDC	Pipeline - ETL		Gajendra tandekar			
CCTech_ACCConnector_CDC_Sync	Job		Gajendra tandekar			
CCTech_ACCConnector_Snapshot_Sync	Job		Gajendra tandekar			

< Previous

Next >

Jobs & Pipelines

Jobs & pipelines

Runs

Send feedback

Name

All

Jobs

Pipelines

Run as

Start: 21-06-2026 19:30

End: 23-06-2026 19:30

Run status

Q Run ID

Error code

Create

Start time	Name	Type	Run as	Launched	Duration	Status	Error code	Run parameters
Jun 23, 2026, 03:28 PM	CCTech_ACCConnector_Snap...	Job	Gajendra tandekar	Manually	31m 14s	Succeeded		manifest_path: /Volumes/...
Jun 23, 2026, 03:29 PM	CCTech_ACCConnector_Bronze	Pipeline - ETL	Gajendra tandekar	bronze_pipeline	30m 16s	Succeeded		

CCTech_ACCConnector_Snapsho...

workspace

Chat

Search data, notebooks, recents, and more...

CTRL + P

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Jobs & Pipelines >

CCTech_ACCConnector_Snapsho...

Send feedback

Run now

Runs

Tasks

Runs

Run total duration

31m 13s

15m 36s

bulk_download

bronze_pipeline

View latest successful run

Cancel runs

Start time	Run ID	Launched	Duration	Status	Error code	Run parameters
Jun 23, 2026, 03:28 PM	131458035228932	Manually	31m 14s	Succeeded		manifest_path: /Volu...

CTech_ACCConnector_Snapsho...

Next

Job ID

196698825933699

Job run ID

131458035228932

Launched

Manually

Started

Jun 23, 2026, 03:28 PM

Ended

Jun 23, 2026, 03:59 PM

Duration

31m 13s

Status

Succeeded

Performance optimized

Enabled

Jun 23

Job details

Job ID

196698825933699

Creator

Gajendra tandekar

Run as

Gajendra tandekar

Description

Add description

Lineage

No lineage information for this job. Learn more

Performance optimized

On

Schedules & Triggers

None

Add trigger

Job parameters

No job parameters are defined for this job

Edit parameters

Compute

Serverless

Swap

Query history

https://dbc-c695526a-8235.cloud.databricks.com/jobs/196698825933699/runs/131458035228932?o=7474649155473442

Work items - Boards

Data Connector - Insight

ACC Connector v2

acc_bronze_volume - Databricks

https://dbc-c695526a-8235.cloud.databricks.com/explore/data/volumes/acc_connector_2_pk_test/bronze/acc_bronze_volume?o=7474649155473442&volumePath=%2FVolumes%2Facc_connector_2_pk_test%2Fbronze%2Facc_bronze_volume%2Fdata_c...

workspace

Chat

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Catalog

Focus: ON

Serverless Starter Warehouse - Serverless

Catalog

Type to search

My organization

workspace

system

acc_connector_1

acc_connector_2_pk_test

bronze

Tables (260)

_meta_bronze_pk_registry

_meta_bronze_schema_versions

_meta_bronze_table_status

admin_account_services

admin_accounts

admin_business_units

admin_companies

admin_project_companies

admin_project_products

admin_project_roles

admin_project_services

admin_project_user_companies

admin_project_user_products

admin_project_user_roles

admin_projects

admin_roles

admin_users

assets_asset_custom_attribute_values

assets_asset_model_sync_records

Catalog Explorer > acc_connector_2_pk_test > bronze >

acc_bronze_volume

Share

Upload to this volume

Overview

Files

Details

Permissions

Description

AI generate

Add

/Volumes/acc_connector_2_pk_test/bronze/acc_bronze_volume / data_connector / 2583e697-0117-415b-bdda-2b04cee0d23 / 2026-06-23T09:55:52

Refresh

Create directory

Filter files and directories at this level

Name	Size	Last modified
README.html	12.15 KB	3 hours ago
_SUCCESS	93.00 B	3 hours ago
_manifest.json	486.88 KB	3 hours ago
admin_account_services.csv	284.00 B	3 hours ago
admin_accounts.csv	141.00 B	3 hours ago
admin_business_units.csv	56.00 B	3 hours ago
admin_companies.csv	350.00 B	3 hours ago
admin_project_companies.csv	182.00 B	3 hours ago
admin_project_products.csv	1.76 KB	3 hours ago
admin_project_roles.csv	504.00 B	3 hours ago
admin_project_services.csv	58.00 B	3 hours ago
admin_project_user_companies.csv	1.36 KB	3 hours ago
admin_project_user_products.csv	9.95 KB	3 hours ago
admin_project_user_roles.csv	2.24 KB	3 hours ago

Genie Code

Run multi-step data and AI tasks

What can Genie Code do?

Run exploratory data analysis

Find data

Connect your tools

Unlock smarter answers, richer context, and faster workflows powered by your organization's knowledge.

Explore connections

for objects, / for commands, ! for history

Agent

Always review the accuracy of responses.

27°C
Light rain

Search

18:55
23-06-2026